Module02_Regularization_Nonlinearities_abalone

March 28, 2026

1 Imports, file paths, and data loading

- This first block prepares the notebook environment and loads the Kaggle files into memory.

[2]:
```
import warnings
from pathlib import Path
from itertools import combinations

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from IPython.display import display, Image

from scipy.stats import shapiro

from sklearn.model_selection import train_test_split, KFold, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, LassoCV, Lasso
from sklearn.decomposition import PCA
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor
from statsmodels.stats.diagnostic import het_breuschpagan, linear_reset,␣
↪linear_rainbow
from statsmodels.stats.stattools import durbin_watson, jarque_bera

warnings.filterwarnings("ignore")

pd.set_option("display.max_columns", 200)
pd.set_option("display.width", 200)
pd.set_option("display.float_format", lambda x: f"{x:,.6f}")

RANDOM_STATE = 42
N_FOLDS = 10
```

```python
TRAIN_PATH = Path(r"F:
↪\NU\predictive_analysis\week2\assignment2\playground-series-s4e4\train.csv")
TEST_PATH = Path(r"F:
↪\NU\predictive_analysis\week2\assignment2\playground-series-s4e4\test.csv")
SAMPLE_SUB_PATH = Path(r"F:
↪\NU\predictive_analysis\week2\assignment2\playground-series-s4e4\sample_submission.
↪csv")

def clean_columns(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()
    df.columns = (
        df.columns
        .str.strip()
        .str.lower()
        .str.replace(r"[^0-9a-zA-Z]+", "_", regex=True)
        .str.strip("_")
    )
    return df

assert TRAIN_PATH.exists(), f"Could not find: {TRAIN_PATH}"
assert TEST_PATH.exists(), f"Could not find: {TEST_PATH}"
assert SAMPLE_SUB_PATH.exists(), f"Could not find: {SAMPLE_SUB_PATH}"

train_raw = clean_columns(pd.read_csv(TRAIN_PATH))
test_raw = clean_columns(pd.read_csv(TEST_PATH))
sample_sub = clean_columns(pd.read_csv(SAMPLE_SUB_PATH))

print("train shape:", train_raw.shape)
print("test shape :", test_raw.shape)
print("sample submission shape:", sample_sub.shape)

display(train_raw.head())
display(test_raw.head())
display(sample_sub.head())
```

```
train shape: (90615, 10)
test shape : (60411, 9)
sample submission shape: (60411, 2)
```

```
   id sex   length  diameter   height  whole_weight  whole_weight_1 ␣
↪whole_weight_2  shell_weight  rings
0   0   F 0.550000  0.430000 0.150000      0.771500        0.328500        0.
↪146500       0.240000      11
1   1   F 0.630000  0.490000 0.145000      1.130000        0.458000        0.
↪276500       0.320000      11
2   2   I 0.160000  0.110000 0.025000      0.021000        0.005500        0.
↪003000       0.005000       6
```

```
3   3   M 0.595000  0.475000 0.150000      0.914500        0.375500        0.
↪205500      0.250000     10
4   4   I 0.555000  0.425000 0.130000      0.782000        0.369500        0.
↪160000      0.197500      9
```

```
     id sex   length  diameter   height  whole_weight  whole_weight_1 ␣
↪whole_weight_2  shell_weight
0  90615   M 0.645000  0.475000 0.155000      1.238000        0.618500        0.
↪312500      0.300500
1  90616   M 0.580000  0.460000 0.160000      0.983000        0.478500        0.
↪219500      0.275000
2  90617   M 0.560000  0.420000 0.140000      0.839500        0.352500        0.
↪184500      0.240500
3  90618   M 0.570000  0.490000 0.145000      0.874000        0.352500        0.
↪186500      0.235000
4  90619   I 0.415000  0.325000 0.110000      0.358000        0.157500        0.
↪067000      0.105000
```

```
     id  rings
0  90615     10
1  90616     10
2  90617     10
3  90618     10
4  90619     10
```

2 Main-effects design matrix

- This block defines the modeling variables and transforms the raw data into a regression-ready

[3]:
```
# Define key columns first
TARGET = "rings"
ID_COL = "id"
CAT_COL = "sex"

# Safety checks
assert TARGET in train_raw.columns, f"{TARGET} not found in train_raw"
assert ID_COL in train_raw.columns, f"{ID_COL} not found in train_raw"
assert ID_COL in test_raw.columns, f"{ID_COL} not found in test_raw"
assert CAT_COL in train_raw.columns, f"{CAT_COL} not found in train_raw"
assert CAT_COL in test_raw.columns, f"{CAT_COL} not found in test_raw"

# Build the list of numeric predictors
numeric_feature_cols = [
    col for col in train_raw.columns
    if col not in [ID_COL, TARGET, CAT_COL]
]
```

```python
# Final feature list used to create the design matrix
base_feature_cols = numeric_feature_cols + [CAT_COL]

def make_design_matrices(train_df: pd.DataFrame, test_df: pd.DataFrame):
    X_train = pd.get_dummies(
        train_df[base_feature_cols],
        columns=[CAT_COL],
        drop_first=True,
        dtype=float
    )

    X_test = pd.get_dummies(
        test_df[base_feature_cols],
        columns=[CAT_COL],
        drop_first=True,
        dtype=float
    )

    # Make sure test has the exact same columns as train
    X_test = X_test.reindex(columns=X_train.columns, fill_value=0.0)

    return X_train.astype(float), X_test.astype(float)

# Create full design matrices
X_full, X_test_full = make_design_matrices(train_raw, test_raw)
y_full = train_raw[TARGET].astype(float).copy()

feature_names = X_full.columns.tolist()

print("Numeric predictors:", numeric_feature_cols)
print("Categorical predictor:", CAT_COL)

print("\nFinal modeling columns:")
display(pd.DataFrame({"feature": feature_names}))

print("X_full shape:", X_full.shape)
print("X_test_full shape:", X_test_full.shape)
print("y_full shape:", y_full.shape)
```

```
Numeric predictors: ['length', 'diameter', 'height', 'whole_weight',
'whole_weight_1', 'whole_weight_2', 'shell_weight']
Categorical predictor: sex

Final modeling columns:

          feature
0          length
1        diameter
```

```
2          height
3    whole_weight
4  whole_weight_1
5  whole_weight_2
6    shell_weight
7           sex_I
8           sex_M
```

```
X_full shape: (90615, 9)
X_test_full shape: (60411, 9)
y_full shape: (90615,)
```

3 Holdout validation split

- This block separates the data into a training portion and a validation portion.

[4]:
```
X_train, X_valid, y_train, y_valid = train_test_split(
    X_full, y_full, test_size=0.20, random_state=RANDOM_STATE
)

X_train = X_train.reset_index(drop=True)
X_valid = X_valid.reset_index(drop=True)
y_train = y_train.reset_index(drop=True)
y_valid = y_valid.reset_index(drop=True)

cv = KFold(n_splits=N_FOLDS, shuffle=True, random_state=RANDOM_STATE)

print("Training set shape   :", X_train.shape)
print("Validation set shape :", X_valid.shape)
```

```
Training set shape   : (72492, 9)
Validation set shape : (18123, 9)
```

4 Helper functions

- This is the analytical toolbox of the notebook. These functions automate the performance summ

[5]:
```
def regression_metrics_table(y_true, y_pred, model_name, split_name,␣
 ↪p_features):
    y_true = np.asarray(y_true, dtype=float)
    y_pred = np.asarray(y_pred, dtype=float)

    n = len(y_true)
    rss = float(np.sum((y_true - y_pred) ** 2))
    mse = rss / n
    rmse = float(np.sqrt(mse))
    mae = float(mean_absolute_error(y_true, y_pred))
    r2 = float(r2_score(y_true, y_pred))
```

```python
    if n - p_features - 1 > 0:
        adj_r2 = float(1 - (1 - r2) * (n - 1) / (n - p_features - 1))
    else:
        adj_r2 = np.nan

    return pd.DataFrame([{
        "model": model_name,
        "split": split_name,
        "n_obs": n,
        "n_predictors": p_features,
        "rss": rss,
        "mse": mse,
        "rmse": rmse,
        "mae": mae,
        "r2": r2,
        "adj_r2": adj_r2
    }])

def coefficient_table(model):
    ci = model.conf_int()
    out = pd.DataFrame({
        "term": model.params.index,
        "coefficient": model.params.values,
        "std_error": model.bse.values,
        "t_value": model.tvalues.values,
        "p_value": model.pvalues.values,
        "ci_lower_95": ci.iloc[:, 0].values,
        "ci_upper_95": ci.iloc[:, 1].values
    })
    return out

def model_fit_summary_table(model, model_name):
    resid = model.resid
    jb_stat, jb_p, skew, kurt = jarque_bera(resid)

    out = pd.DataFrame([{
        "model": model_name,
        "n_obs": float(model.nobs),
        "df_model": float(model.df_model),
        "df_resid": float(model.df_resid),
        "rss": float(np.sum(resid ** 2)),
        "mse_resid": float(model.mse_resid),
        "rmse_resid": float(np.sqrt(model.mse_resid)),
        "r2": float(model.rsquared),
        "adj_r2": float(model.rsquared_adj),
```

```
        "f_statistic": float(model.fvalue) if model.fvalue is not None else np.
↪nan,
        "f_pvalue": float(model.f_pvalue) if model.f_pvalue is not None else np.
↪nan,
        "aic": float(model.aic),
        "bic": float(model.bic),
        "durbin_watson": float(durbin_watson(resid)),
        "jarque_bera_stat": float(jb_stat),
        "jarque_bera_pvalue": float(jb_p),
        "skew": float(skew),
        "kurtosis": float(kurt),
        "condition_number": float(model.condition_number)
    }])
    return out

def vif_table(X_df: pd.DataFrame):
    X_df = X_df.copy()
    vif_rows = []
    for i, col in enumerate(X_df.columns):
        vif_value = variance_inflation_factor(X_df.values, i)
        vif_rows.append({"feature": col, "vif": float(vif_value)})
    return pd.DataFrame(vif_rows).sort_values("vif", ascending=False).
↪reset_index(drop=True)

def influence_table(model, top_n=20):
    infl = model.get_influence()
    leverage = infl.hat_matrix_diag
    studentized = infl.resid_studentized_external
    cooks_d = infl.cooks_distance[0]

    n = int(model.nobs)
    p = int(model.df_model) + 1  # include intercept
    leverage_cutoff = 2 * p / n
    cooks_cutoff = 4 / n

    out = pd.DataFrame({
        "observation": np.arange(n),
        "fitted": model.fittedvalues,
        "residual": model.resid,
        "studentized_residual": studentized,
        "leverage": leverage,
        "cooks_distance": cooks_d,
        "high_leverage_flag": leverage > leverage_cutoff,
        "high_cooks_flag": cooks_d > cooks_cutoff,
        "outlier_flag_abs_studentized_gt_3": np.abs(studentized) > 3
    })
```

```
    out = out.sort_values(["cooks_distance", "leverage"], ascending=[False,␣
↪False]).reset_index(drop=True)
    return out.head(top_n), leverage_cutoff, cooks_cutoff

def assumption_summary_table(model, X_predictors: pd.DataFrame, model_name:␣
↪str):
    resid = model.resid
    n = int(model.nobs)
    p = int(model.df_model) + 1  # include intercept

    # Homoscedasticity
    bp_stat, bp_pvalue, _, _ = het_breuschpagan(resid, model.model.exog)

    # Normality
    jb_stat, jb_pvalue, skew, kurt = jarque_bera(resid)

    # Shapiro is safe here because the training sample is not huge
    shapiro_stat, shapiro_pvalue = shapiro(resid)

    # Independence
    dw_stat = durbin_watson(resid)

    # Linearity checks
    try:
        reset_res = linear_reset(model, power=2, use_f=True)
        reset_stat = float(reset_res.fvalue)
        reset_pvalue = float(reset_res.pvalue)
    except Exception:
        reset_stat = np.nan
        reset_pvalue = np.nan

    try:
        rainbow_stat, rainbow_pvalue = linear_rainbow(model)
    except Exception:
        rainbow_stat, rainbow_pvalue = np.nan, np.nan

    # Collinearity
    if X_predictors.shape[1] >= 2:
        vif_df = vif_table(X_predictors)
        max_vif = float(vif_df["vif"].replace([np.inf, -np.inf], np.nan).max())
    else:
        max_vif = np.nan

    # Influence
    infl = model.get_influence()
    leverage = infl.hat_matrix_diag
    cooks_d = infl.cooks_distance[0]
```

```
    leverage_cutoff = 2 * p / n
    cooks_cutoff = 4 / n

    out = pd.DataFrame([
        {
            "model": model_name,
            "assumption": "Linearity",
            "check": "RESET test p-value",
            "value": reset_pvalue,
            "guideline": "p > 0.05 is desirable",
            "result": "Pass" if pd.notnull(reset_pvalue) and reset_pvalue > 0.
↪05 else "Review"
        },
        {
            "model": model_name,
            "assumption": "Linearity",
            "check": "Rainbow test p-value",
            "value": rainbow_pvalue,
            "guideline": "p > 0.05 is desirable",
            "result": "Pass" if pd.notnull(rainbow_pvalue) and rainbow_pvalue >␣
↪0.05 else "Review"
        },
        {
            "model": model_name,
            "assumption": "Independence of errors",
            "check": "Durbin-Watson statistic",
            "value": dw_stat,
            "guideline": "Around 2 is desirable",
            "result": "Pass" if 1.5 <= dw_stat <= 2.5 else "Review"
        },
        {
            "model": model_name,
            "assumption": "Homoscedasticity",
            "check": "Breusch-Pagan p-value",
            "value": bp_pvalue,
            "guideline": "p > 0.05 is desirable",
            "result": "Pass" if bp_pvalue > 0.05 else "Review"
        },
        {
            "model": model_name,
            "assumption": "Normality of residuals",
            "check": "Jarque-Bera p-value",
            "value": jb_pvalue,
            "guideline": "p > 0.05 is desirable",
            "result": "Pass" if jb_pvalue > 0.05 else "Review"
        },
        {
```

```
            "model": model_name,
            "assumption": "Normality of residuals",
            "check": "Shapiro-Wilk p-value",
            "value": shapiro_pvalue,
            "guideline": "p > 0.05 is desirable",
            "result": "Pass" if shapiro_pvalue > 0.05 else "Review"
        },
        {
            "model": model_name,
            "assumption": "Collinearity",
            "check": "Maximum VIF",
            "value": max_vif,
            "guideline": "Below 5 is desirable",
            "result": "Pass" if pd.notnull(max_vif) and max_vif < 5 else␣
↪"Review"
        },
        {
            "model": model_name,
            "assumption": "Influential observations",
            "check": "Count Cook's D > 4/n",
            "value": int(np.sum(cooks_d > cooks_cutoff)),
            "guideline": "Smaller is desirable",
            "result": "Review"
        },
        {
            "model": model_name,
            "assumption": "High leverage observations",
            "check": "Count leverage > 2p/n",
            "value": int(np.sum(leverage > leverage_cutoff)),
            "guideline": "Smaller is desirable",
            "result": "Review"
        }
    ])
    return out

def plot_ols_diagnostics(model, title):
    fitted = model.fittedvalues
    resid = model.resid
    infl = model.get_influence()
    leverage = infl.hat_matrix_diag
    studentized = infl.resid_studentized_external

    fig, axes = plt.subplots(2, 2, figsize=(12, 10))

    # Residuals vs fitted
    axes[0, 0].scatter(fitted, resid, alpha=0.7)
    axes[0, 0].axhline(0, linestyle="--")
```

```
    axes[0, 0].set_title("Residuals vs Fitted")
    axes[0, 0].set_xlabel("Fitted values")
    axes[0, 0].set_ylabel("Residuals")

    # Q-Q plot
    sm.qqplot(resid, line="45", ax=axes[0, 1])
    axes[0, 1].set_title("Normal Q-Q")

    # Scale-location
    axes[1, 0].scatter(fitted, np.sqrt(np.abs(studentized)), alpha=0.7)
    axes[1, 0].set_title("Scale-Location")
    axes[1, 0].set_xlabel("Fitted values")
    axes[1, 0].set_ylabel("Sqrt(|Studentized residual|)")

    # Residuals vs leverage
    axes[1, 1].scatter(leverage, studentized, alpha=0.7)
    axes[1, 1].axhline(0, linestyle="--")
    axes[1, 1].set_title("Residuals vs Leverage")
    axes[1, 1].set_xlabel("Leverage")
    axes[1, 1].set_ylabel("Studentized residuals")

    fig.suptitle(title, fontsize=14)
    plt.tight_layout()
    plt.show()
```

5 Baseline comparison models

- This block establishes reference models before the more advanced models are built.

[6]:
```
# Null model
null_pred_valid = np.repeat(y_train.mean(), len(y_valid))

# Full OLS model with all main effects
full_ols_train = sm.OLS(y_train, sm.add_constant(X_train, has_constant="add")).
 ↪fit()
full_ols_valid_pred = full_ols_train.predict(sm.add_constant(X_valid,␣
 ↪has_constant="add"))

baseline_comparison = pd.concat([
    regression_metrics_table(y_valid, null_pred_valid, "Null mean model",␣
 ↪"validation", 0),
    regression_metrics_table(y_valid, full_ols_valid_pred, "Full OLS␣
 ↪main-effects model", "validation", X_train.shape[1])
], ignore_index=True)

display(baseline_comparison)
```

```
                        model       split  n_obs  n_predictors            rss ␣
↪     mse      rmse       mae        r2    adj_r2
0              Null mean model  validation  18123             0 186,165.952688␣
↪10.272358 3.205052 2.349198 -0.000000 -0.000000
1  Full OLS main-effects model  validation  18123             9  74,219.532043 ␣
↪4.095323 2.023690 1.393015  0.601326  0.601128
```

6. A.1 LassoCV for regularization-informed subset selection

[7]:
```
scaler_lasso = StandardScaler()
X_train_scaled = scaler_lasso.fit_transform(X_train)

lasso_cv = LassoCV(
    cv=cv,
    max_iter=50000,
    random_state=RANDOM_STATE
)
lasso_cv.fit(X_train_scaled, y_train)

lasso_mean_mse = lasso_cv.mse_path_.mean(axis=1)
lasso_se_mse = lasso_cv.mse_path_.std(axis=1, ddof=1) / np.sqrt(N_FOLDS)

idx_min = int(np.argmin(lasso_mean_mse))
alpha_min = float(lasso_cv.alphas_[idx_min])

threshold_1se = lasso_mean_mse[idx_min] + lasso_se_mse[idx_min]
eligible_idx = np.where(lasso_mean_mse <= threshold_1se)[0]
idx_1se = int(eligible_idx[0])  # largest alpha within 1-SE because alphas are␣
↪descending
alpha_1se = float(lasso_cv.alphas_[idx_1se])

lasso_min = Lasso(alpha=alpha_min, max_iter=50000)
lasso_min.fit(X_train_scaled, y_train)

lasso_1se = Lasso(alpha=alpha_1se, max_iter=50000)
lasso_1se.fit(X_train_scaled, y_train)

lasso_cv_table = pd.DataFrame({
    "alpha": lasso_cv.alphas_,
    "mean_cv_mse": lasso_mean_mse,
    "cv_mse_se": lasso_se_mse,
    "mean_cv_rmse": np.sqrt(lasso_mean_mse),
    "selected_alpha_min": lasso_cv.alphas_ == alpha_min,
    "selected_alpha_1se": lasso_cv.alphas_ == alpha_1se
}).sort_values("alpha", ascending=False).reset_index(drop=True)

lasso_screening_table = pd.DataFrame({
    "feature": feature_names,
```

```python
    "coef_alpha_min": lasso_min.coef_,
    "abs_coef_alpha_min": np.abs(lasso_min.coef_),
    "coef_alpha_1se": lasso_1se.coef_,
    "abs_coef_alpha_1se": np.abs(lasso_1se.coef_)
}).sort_values("abs_coef_alpha_1se", ascending=False).reset_index(drop=True)

candidate_vars = lasso_screening_table.loc[
    np.abs(lasso_screening_table["coef_alpha_1se"]) > 1e-10, "feature"
].tolist()

if len(candidate_vars) == 0:
    candidate_vars = lasso_screening_table.loc[
        np.abs(lasso_screening_table["coef_alpha_min"]) > 1e-10, "feature"
    ].tolist()

if len(candidate_vars) == 0:
    candidate_vars = feature_names.copy()

lasso_screening_summary = pd.DataFrame([{
    "alpha_min": alpha_min,
    "alpha_1se": alpha_1se,
    "n_features_alpha_min": int(np.sum(np.abs(lasso_min.coef_) > 1e-10)),
    "n_features_alpha_1se": int(np.sum(np.abs(lasso_1se.coef_) > 1e-10)),
    "candidate_variables_for_subset_search": ", ".join(candidate_vars)
}])

print("Candidate variables selected for subset search:")
print(candidate_vars)

display(lasso_screening_summary)
display(lasso_screening_table)
```

```
Candidate variables selected for subset search:
['shell_weight', 'whole_weight_1', 'height', 'diameter', 'sex_I',
'whole_weight', 'whole_weight_2']
```

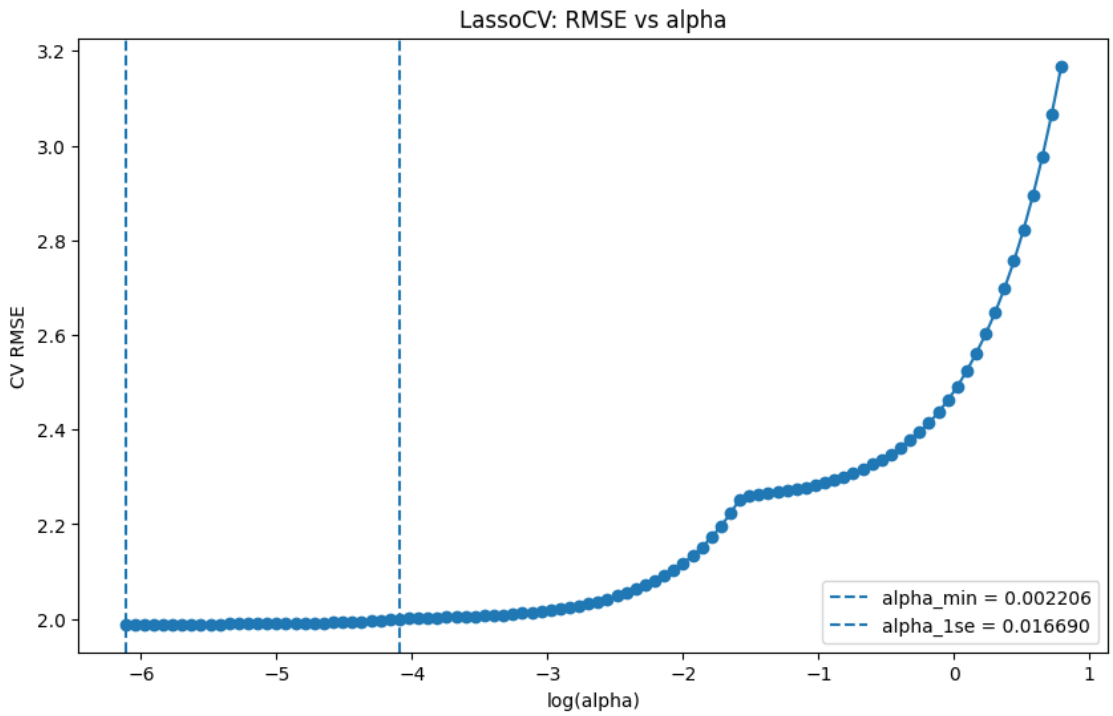
```
   alpha_min  alpha_1se  n_features_alpha_min  n_features_alpha_1se            ␣
 ↪candidate_variables_for_subset_search
0   0.002206   0.016690                     9                     7 ␣
 ↪shell_weight, whole_weight_1, height, diameter…

          feature  coef_alpha_min  abs_coef_alpha_min  coef_alpha_1se ␣
 ↪abs_coef_alpha_1se
0    shell_weight        2.778902            2.778902        3.006237           ␣
 ↪3.006237
1  whole_weight_1       -3.140700            3.140700       -2.563760           ␣
 ↪2.563760
```

```
2          height        0.753473            0.753473        0.758066
↪0.758066
3        diameter        0.572922            0.572922        0.502723
↪0.502723
4           sex_I       -0.340432            0.340432       -0.327177
↪0.327177
5    whole_weight        1.489079            1.489079        0.223503
↪0.223503
6  whole_weight_2       -0.604988            0.604988       -0.054848
↪0.054848
7          length       -0.004216            0.004216        0.000000
↪0.000000
8           sex_M       -0.006242            0.006242        0.000000
↪0.000000
```

6 Lasso cross-validation curve

This plot visualizes how model error changes as the lasso penalty changes.

[8]:
```
plt.figure(figsize=(10, 6))
plt.plot(np.log(lasso_cv_table["alpha"]), lasso_cv_table["mean_cv_rmse"],␣
↪marker="o")
plt.axvline(np.log(alpha_min), linestyle="--", label=f"alpha_min = {alpha_min:.
↪6f}")
plt.axvline(np.log(alpha_1se), linestyle="--", label=f"alpha_1se = {alpha_1se:.
↪6f}")
plt.xlabel("log(alpha)")
plt.ylabel("CV RMSE")
plt.title("LassoCV: RMSE vs alpha")
plt.legend()
plt.show()
```



7 Exhaustive best subset search on screened variables

[9]:
```
def evaluate_subset(X_df, y_series, subset_vars, sigma2_full_model,␣
↪cv_splitter):
    subset_vars = list(subset_vars)
    X_sub = X_df[subset_vars].copy()
    X_sub_const = sm.add_constant(X_sub, has_constant="add")

    model = sm.OLS(y_series, X_sub_const).fit()

    n = X_sub_const.shape[0]
    d = X_sub_const.shape[1]  # includes intercept
    rss = float(np.sum(model.resid ** 2))

    neg_mse_scores = cross_val_score(
        LinearRegression(),
        X_sub,
        y_series,
        cv=cv_splitter,
        scoring="neg_mean_squared_error"
    )
    rmse_folds = np.sqrt(-neg_mse_scores)
```

```
    return {
        "subset_size": len(subset_vars),
        "predictors": " + ".join(subset_vars),
        "subset_tuple": tuple(subset_vars),
        "rss": rss,
        "r2": float(model.rsquared),
        "adj_r2": float(model.rsquared_adj),
        "aic": float(model.aic),
        "bic": float(model.bic),
        "cp": float((rss + 2 * d * sigma2_full_model) / n),
        "cv_rmse_mean": float(rmse_folds.mean()),
        "cv_rmse_se": float(rmse_folds.std(ddof=1) / np.sqrt(cv_splitter.
 ↪get_n_splits())),
        "cv_rmse_min_fold": float(rmse_folds.min()),
        "cv_rmse_max_fold": float(rmse_folds.max()),
        "n_parameters_including_intercept": d
    }

full_model_for_cp = sm.OLS(y_train, sm.add_constant(X_train,␣
 ↪has_constant="add")).fit()
sigma2_full_model = float(full_model_for_cp.mse_resid)

subset_rows = []
for k in range(1, len(candidate_vars) + 1):
    for combo in combinations(candidate_vars, k):
        subset_rows.append(
            evaluate_subset(X_train, y_train, combo, sigma2_full_model, cv)
        )

subset_search_table = pd.DataFrame(subset_rows).sort_values(
    ["subset_size", "cv_rmse_mean", "bic", "aic"]
).reset_index(drop=True)

# Best model at each size
subset_frontier_table = (
    subset_search_table
    .sort_values(["subset_size", "cv_rmse_mean", "bic", "aic"])
    .groupby("subset_size", as_index=False)
    .first()
)

# One-standard-error rule on the frontier
frontier_best_idx = subset_frontier_table["cv_rmse_mean"].idxmin()
frontier_min_rmse = subset_frontier_table.loc[frontier_best_idx, "cv_rmse_mean"]
frontier_min_se = subset_frontier_table.loc[frontier_best_idx, "cv_rmse_se"]

eligible_frontier = subset_frontier_table.loc[
```

```
    subset_frontier_table["cv_rmse_mean"] <= frontier_min_rmse + frontier_min_se
].copy()

selected_subset_row = eligible_frontier.sort_values(
    ["subset_size", "bic", "aic", "cv_rmse_mean"]
).iloc[0]

selected_subset_vars = list(selected_subset_row["subset_tuple"])

subset_selection_summary = pd.DataFrame([{
    "selection_rule": "10-fold CV + one-standard-error rule on best subset␣
 ↪frontier",
    "selected_subset_size": int(selected_subset_row["subset_size"]),
    "selected_predictors": " + ".join(selected_subset_vars),
    "selected_cv_rmse_mean": float(selected_subset_row["cv_rmse_mean"]),
    "selected_cv_rmse_se": float(selected_subset_row["cv_rmse_se"]),
    "selected_bic": float(selected_subset_row["bic"]),
    "selected_cp": float(selected_subset_row["cp"])
}])

print("Selected subset predictors:")
print(selected_subset_vars)

display(subset_selection_summary)
display(subset_frontier_table)
```

```
Selected subset predictors:
['shell_weight', 'whole_weight_1', 'height', 'diameter', 'sex_I']

                                     selection_rule  selected_subset_size      ␣
 ↪                        selected_predictors  selected_cv_rmse_mean ␣
 ↪selected_cv_rmse_se   selected_bic  selected_cp
0  10-fold CV + one-standard-error rule on best s…                     5 ␣
 ↪shell_weight + whole_weight_1 + height + diame…               1.999227        ␣
 ↪    0.011299 306,213.480482     3.996666

   subset_size                                          predictors              ␣
 ↪                       subset_tuple            rss       r2   adj_r2          ␣
 ↪ aic            bic       cp  \
0            1                                       shell_weight              ␣
 ↪                    (shell_weight,) 375,084.422807 0.484762 0.484755 324,880.
 ↪666394 324,899.048857 5.174367
1            2                       shell_weight + whole_weight_1              ␣
 ↪     (shell_weight, whole_weight_1) 314,412.885039 0.568104 0.568092 312,091.
 ↪881751 312,119.455446 4.337535
2            3              shell_weight + whole_weight_1 + height            ␣
 ↪(shell_weight, whole_weight_1, height) 298,097.097029 0.590516 0.590499␣
 ↪308,230.943984 308,267.708910 4.112574
```

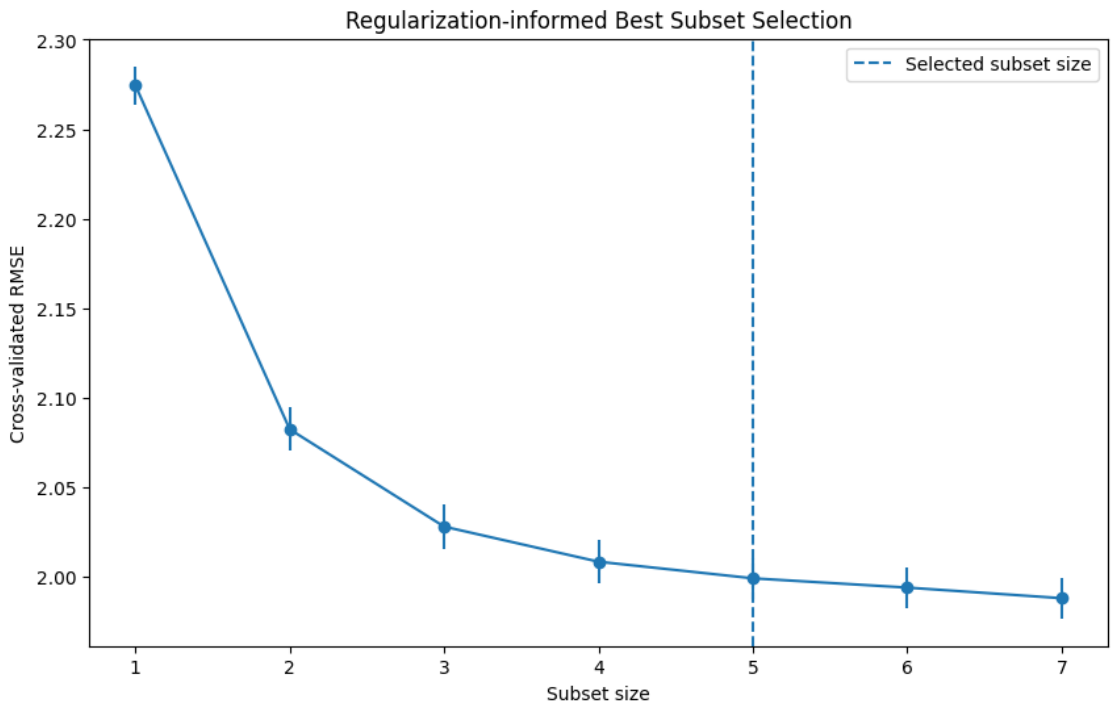
```
3            4     shell_weight + whole_weight_1 + height + sex_I
↪(shell_weight, whole_weight_1, height, sex_I) 292,368.074535 0.598386 0.598364
↪306,826.184189 306,872.140347 4.033653
4            5  shell_weight + whole_weight_1 + height + diame…  (shell_weight,
↪whole_weight_1, height, diamete… 289,678.936051 0.602080 0.602052 306,158.
↪333093 306,213.480482 3.996666
5            6  shell_weight + whole_weight_1 + height + diame…  (shell_weight,
↪whole_weight_1, height, diamete… 288,150.821244 0.604179 0.604146 305,776.
↪910967 305,841.249588 3.975696
6            7  shell_weight + whole_weight_1 + height + diame…  (shell_weight,
↪whole_weight_1, height, diamete… 286,433.925254 0.606537 0.606499 305,345.
↪688209 305,419.218061 3.952121

   cv_rmse_mean  cv_rmse_se  cv_rmse_min_fold  cv_rmse_max_fold
↪n_parameters_including_intercept
0      2.274518    0.010625          2.221415          2.333671
↪                2
1      2.082444    0.012104          2.027467          2.157642
↪                3
2      2.028107    0.012572          1.969907          2.112473
↪                4
3      2.008498    0.011978          1.954759          2.087688
↪                5
4      1.999227    0.011299          1.949348          2.069208
↪                6
5      1.994048    0.011183          1.948316          2.058972
↪                7
6      1.988127    0.011267          1.942674          2.053999
↪                8
```

8 Best subset frontier plot

- This plot shows the best cross-validated RMSE achieved at each subset size.

[10]:
```
plt.figure(figsize=(10, 6))
plt.errorbar(
    subset_frontier_table["subset_size"],
    subset_frontier_table["cv_rmse_mean"],
    yerr=subset_frontier_table["cv_rmse_se"],
    marker="o"
)
plt.axvline(selected_subset_row["subset_size"], linestyle="--", label="Selected
↪subset size")
plt.xlabel("Subset size")
plt.ylabel("Cross-validated RMSE")
plt.title("Regularization-informed Best Subset Selection")
```

```
plt.legend()
plt.show()
```



9 Final OLS subset model and statistical tables

- This block fits the final regularization-informed subset model on the training data and creat

[11]:
```
def influence_table_fast(model, top_n=20):
    infl = model.get_influence()

    leverage = np.asarray(infl.hat_matrix_diag)

    # Fast version: use internal studentized residuals
    # instead of external studentized residuals
    try:
        studentized = np.asarray(infl.resid_studentized_internal)
    except Exception:
        studentized = np.asarray(
            model.resid / np.sqrt(model.mse_resid * (1 - leverage))
        )

    cooks_d = np.asarray(infl.cooks_distance[0])
```

```
    n = int(model.nobs)
    p = int(model.df_model) + 1  # include intercept
    leverage_cutoff = 2 * p / n
    cooks_cutoff = 4 / n

    out = pd.DataFrame({
        "observation": np.arange(n),
        "fitted": np.asarray(model.fittedvalues),
        "residual": np.asarray(model.resid),
        "studentized_residual_internal": studentized,
        "leverage": leverage,
        "cooks_distance": cooks_d,
        "high_leverage_flag": leverage > leverage_cutoff,
        "high_cooks_flag": cooks_d > cooks_cutoff,
        "outlier_flag_abs_studentized_gt_3": np.abs(studentized) > 3
    })

    out = out.sort_values(
        ["cooks_distance", "leverage"],
        ascending=[False, False]
    ).reset_index(drop=True)

    return out.head(top_n), leverage_cutoff, cooks_cutoff


# Build subset design matrices once
X_train_subset = X_train[selected_subset_vars].copy()
X_valid_subset = X_valid[selected_subset_vars].copy()

X_train_subset_const = sm.add_constant(X_train_subset, has_constant="add")
X_valid_subset_const = sm.add_constant(X_valid_subset, has_constant="add")

# Fit OLS subset model
subset_model = sm.OLS(y_train, X_train_subset_const).fit()

# Predictions
subset_train_pred = subset_model.predict(X_train_subset_const)
subset_valid_pred = subset_model.predict(X_valid_subset_const)

# Statistical tables
subset_coef_table = coefficient_table(subset_model)

subset_fit_summary = model_fit_summary_table(
    subset_model,
    "Regularization-informed best subset OLS"
)
```

```python
subset_vif_table = vif_table(X_train_subset)

subset_assumption_table = assumption_summary_table(
    subset_model,
    X_train_subset,
    "Regularization-informed best subset OLS"
)

# Fast influence table
subset_influence_table, subset_leverage_cutoff, subset_cooks_cutoff =␣
 ↪influence_table_fast(
    subset_model,
    top_n=20
)

# Performance table
subset_metrics_table = pd.concat([
    regression_metrics_table(
        y_train,
        subset_train_pred,
        "Regularization-informed best subset OLS",
        "train",
        len(selected_subset_vars)
    ),
    regression_metrics_table(
        y_valid,
        subset_valid_pred,
        "Regularization-informed best subset OLS",
        "validation",
        len(selected_subset_vars)
    )
], ignore_index=True)

print(f"Subset leverage cutoff: {subset_leverage_cutoff:.6f}")
print(f"Subset Cook's D cutoff: {subset_cooks_cutoff:.6f}")

display(subset_fit_summary)
display(subset_coef_table)
display(subset_vif_table)
display(subset_assumption_table)
display(subset_metrics_table)
display(subset_influence_table)
```

```
Subset leverage cutoff: 0.000166
Subset Cook's D cutoff: 0.000055
```

```
                                    model         n_obs  df_model      df_resid␣
↪           rss  mse_resid  rmse_resid       r2   adj_r2   f_statistic ␣
↪f_pvalue            aic            bic  \
0  Regularization-informed best subset OLS 72,492.000000  5.000000 72,486.000000␣
↪289,678.936051   3.996343    1.999086 0.602080 0.602052 21,935.229925  0.
↪000000 306,158.333093 306,213.480482

  durbin_watson  jarque_bera_stat  jarque_bera_pvalue     skew  kurtosis ␣
↪condition_number
0       1.994440    159,780.033945            0.000000 1.462654  9.658912       ␣
↪ 90.246902

            term  coefficient  std_error     t_value  p_value  ci_lower_95 ␣
↪ci_upper_95
0          const     3.644251   0.062749   58.076215 0.000000     3.521263    ␣
↪3.767240
1   shell_weight    24.625457   0.174027  141.503928 0.000000    24.284365    ␣
↪24.966548
2  whole_weight_1   -13.298888   0.098480 -135.041462 0.000000   -13.491909   ␣
↪-13.105868
3         height    19.962148   0.530488   37.629787 0.000000    18.922394    ␣
↪21.001903
4       diameter     6.404928   0.246910   25.940312 0.000000     5.920985    ␣
↪6.888871
5          sex_I    -0.698711   0.020841  -33.525376 0.000000    -0.739560    ␣
↪-0.657863

         feature       vif
0       diameter 97.349244
1         height 96.857233
2   shell_weight 32.369732
3  whole_weight_1 24.607574
4          sex_I  2.176087

                                    model                  assumption          ␣
↪         check        value              guideline  result
0  Regularization-informed best subset OLS                   Linearity      ␣
↪RESET test p-value     0.000000  p > 0.05 is desirable  Review
1  Regularization-informed best subset OLS                   Linearity    ␣
↪Rainbow test p-value     0.979006  p > 0.05 is desirable    Pass
2  Regularization-informed best subset OLS      Independence of errors ␣
↪Durbin-Watson statistic     1.994440  Around 2 is desirable    Pass
3  Regularization-informed best subset OLS            Homoscedasticity   ␣
↪Breusch-Pagan p-value     0.000000  p > 0.05 is desirable  Review
4  Regularization-informed best subset OLS      Normality of residuals     ␣
↪Jarque-Bera p-value     0.000000  p > 0.05 is desirable  Review
5  Regularization-informed best subset OLS      Normality of residuals    ␣
↪Shapiro-Wilk p-value     0.000000  p > 0.05 is desirable  Review
```

```
6  Regularization-informed best subset OLS                Collinearity
↪   Maximum VIF    97.349244   Below 5 is desirable  Review
7  Regularization-informed best subset OLS    Influential observations     Count
↪Cook's D > 4/n 4,380.000000   Smaller is desirable  Review
8  Regularization-informed best subset OLS  High leverage observations    Count
↪leverage > 2p/n 5,063.000000   Smaller is desirable  Review

                                    model       split  n_obs  n_predictors
↪      rss      mse     rmse      mae       r2   adj_r2
0  Regularization-informed best subset OLS       train  72492             5
↪289,678.936051 3.996012 1.999003 1.374227 0.602080 0.602052
1  Regularization-informed best subset OLS  validation  18123             5
↪75,281.179636 4.153903 2.038113 1.400566 0.595623 0.595512

   observation    fitted   residual  studentized_residual_internal  leverage
↪cooks_distance  high_leverage_flag  high_cooks_flag
↪outlier_flag_abs_studentized_gt_3
0         10362 29.145309 -21.145309                     -10.977222  0.071503
↪    1.546603                True             True
↪ True
1         60373 29.216340 -19.216340                      -9.953528  0.067338
↪    1.192165                True             True
↪ True
2         53381 26.513393 -18.513393                      -9.503840  0.050465
↪    0.800066                True             True
↪ True
3         60496 27.495344 -18.495344                      -9.263594  0.002523
↪    0.036170                True             True
↪ True
4         36375 20.387564 -10.387564                      -5.214793  0.007134
↪    0.032567                True             True
↪ True
5         53748 20.290542 -10.290542                      -5.164134  0.006384
↪    0.028557                True             True
↪ True
6         66670 21.729916  -9.729916                      -4.884407  0.007040
↪    0.028192                True             True
↪ True
7         27584 18.868860  -7.868860                      -3.951219  0.007573
↪    0.019854                True             True
↪ True
8         18468 23.913004 -14.913004                      -7.465138  0.001400
↪    0.013017                True             True
↪ True
9         25431 23.505958 -12.505958                      -6.260311  0.001428
↪    0.009343                True             True
↪ True
```

```
10         2538 22.368297 -11.368297                      -5.690882  0.001452
    0.007850                True             True
True
11        46525  9.216381  19.783619                       9.898649  0.000468
    0.007641                True             True
True
12        53920 24.212436 -11.212436                      -5.612642  0.001375
    0.007228                True             True
True
13        67534 21.802316 -10.802316                      -5.406532  0.001074
    0.005237                True             True
True
14        56911 21.977692  -9.977692                      -4.994253  0.001251
    0.005206                True             True
True
15        35575 21.811158  -8.811158                      -4.411025  0.001555
    0.005051                True             True
True
16         1404 21.857842  -8.857842                      -4.434188  0.001462
    0.004797                True             True
True
17        16332  4.542705   8.457295                       4.233678  0.001462
    0.004374                True             True
True
18         2778 20.311111  -9.311111                      -4.660442  0.001183
    0.004287                True             True
True
19        41219 24.980582  -7.980582                      -3.995281  0.001584
    0.004220                True             True
True
```

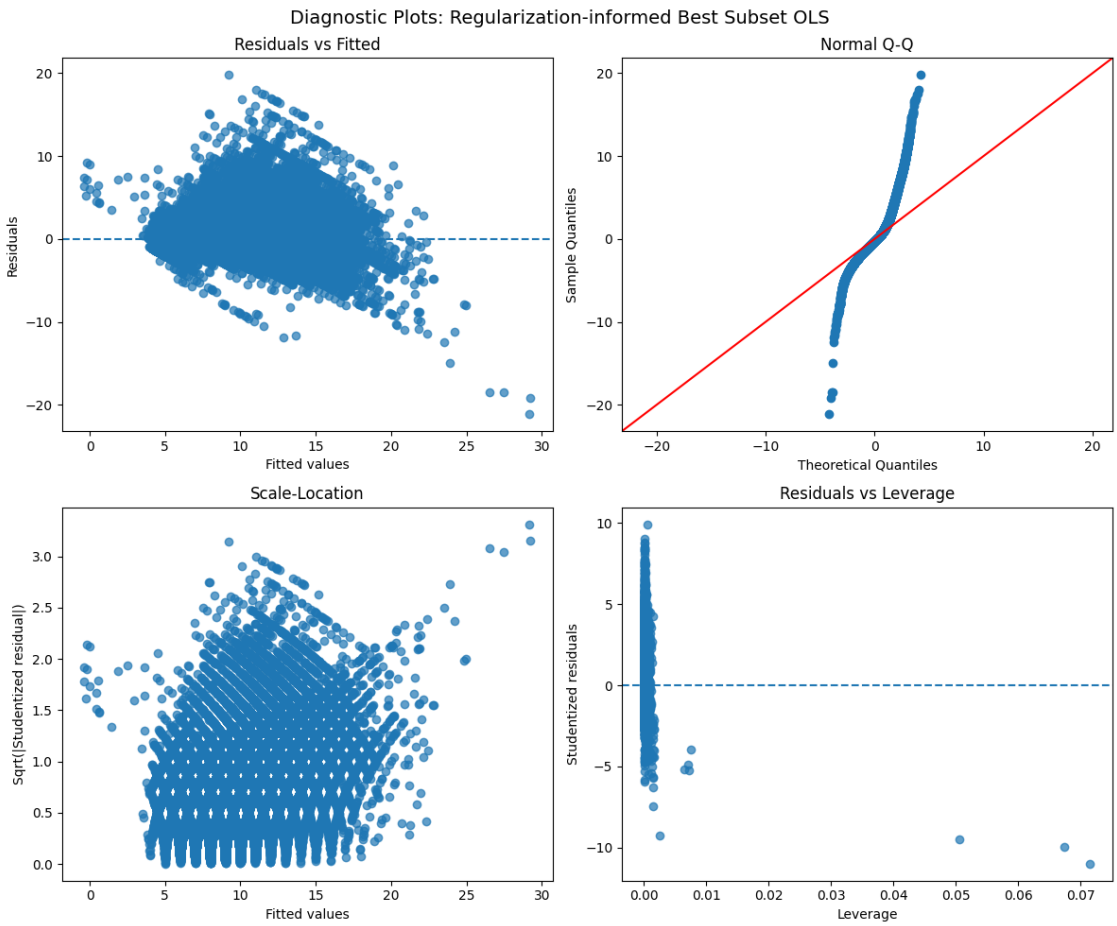
10 Faster diagnostic plots for subset model

- This block creates the residual plots for the subset model, but with a faster implementation

[12]:
```
def plot_ols_diagnostics_fast(model, title):
    fitted = np.asarray(model.fittedvalues)
    resid = np.asarray(model.resid)

    infl = model.get_influence()
    leverage = np.asarray(infl.hat_matrix_diag)

    # Fast studentized residuals
    try:
        studentized = np.asarray(infl.resid_studentized_internal)
    except Exception:
        studentized = resid / np.sqrt(model.mse_resid * (1 - leverage))
```

```
    fig, axes = plt.subplots(2, 2, figsize=(12, 10))

    # 1. Residuals vs Fitted
    axes[0, 0].scatter(fitted, resid, alpha=0.7)
    axes[0, 0].axhline(0, linestyle="--")
    axes[0, 0].set_title("Residuals vs Fitted")
    axes[0, 0].set_xlabel("Fitted values")
    axes[0, 0].set_ylabel("Residuals")

    # 2. Normal Q-Q
    sm.qqplot(resid, line="45", ax=axes[0, 1])
    axes[0, 1].set_title("Normal Q-Q")

    # 3. Scale-Location
    axes[1, 0].scatter(fitted, np.sqrt(np.abs(studentized)), alpha=0.7)
    axes[1, 0].set_title("Scale-Location")
    axes[1, 0].set_xlabel("Fitted values")
    axes[1, 0].set_ylabel("Sqrt(|Studentized residual|)")

    # 4. Residuals vs Leverage
    axes[1, 1].scatter(leverage, studentized, alpha=0.7)
    axes[1, 1].axhline(0, linestyle="--")
    axes[1, 1].set_title("Residuals vs Leverage")
    axes[1, 1].set_xlabel("Leverage")
    axes[1, 1].set_ylabel("Studentized residuals")

    fig.suptitle(title, fontsize=14)
    plt.tight_layout()
    plt.show()


print(f"Subset leverage cutoff = {subset_leverage_cutoff:.6f}")
print(f"Subset Cook's D cutoff = {subset_cooks_cutoff:.6f}")

plot_ols_diagnostics_fast(
    subset_model,
    "Diagnostic Plots: Regularization-informed Best Subset OLS"
)
```

```
Subset leverage cutoff = 0.000166
Subset Cook's D cutoff = 0.000055
```



11 Principal Components Regression selection

[13]:
```
pcr_cv_rows = []

for n_comp in range(1, X_train.shape[1] + 1):
    pcr_pipe = Pipeline([
        ("scaler", StandardScaler()),
        ("pca", PCA(n_components=n_comp)),
        ("linreg", LinearRegression())
    ])

    neg_mse_scores = cross_val_score(
        pcr_pipe, X_train, y_train,
        cv=cv,
        scoring="neg_mean_squared_error"
    )
    r2_scores = cross_val_score(
```

```
        pcr_pipe, X_train, y_train,
        cv=cv,
        scoring="r2"
    )

    rmse_folds = np.sqrt(-neg_mse_scores)

    pcr_cv_rows.append({
        "n_components": n_comp,
        "cv_rmse_mean": float(rmse_folds.mean()),
        "cv_rmse_se": float(rmse_folds.std(ddof=1) / np.sqrt(N_FOLDS)),
        "cv_r2_mean": float(r2_scores.mean()),
        "cv_r2_se": float(r2_scores.std(ddof=1) / np.sqrt(N_FOLDS))
    })

pcr_cv_table = pd.DataFrame(pcr_cv_rows)

pcr_best_idx = pcr_cv_table["cv_rmse_mean"].idxmin()
pcr_min_rmse = pcr_cv_table.loc[pcr_best_idx, "cv_rmse_mean"]
pcr_min_se = pcr_cv_table.loc[pcr_best_idx, "cv_rmse_se"]

pcr_eligible = pcr_cv_table.loc[
    pcr_cv_table["cv_rmse_mean"] <= pcr_min_rmse + pcr_min_se
].copy()

selected_n_components = int(pcr_eligible.sort_values("n_components").
 ↪iloc[0]["n_components"])

pcr_selection_summary = pd.DataFrame([{
    "selection_rule": "10-fold CV + one-standard-error rule",
    "selected_n_components": selected_n_components,
    "selected_cv_rmse_mean": float(
        pcr_cv_table.loc[pcr_cv_table["n_components"] == selected_n_components,␣
 ↪"cv_rmse_mean"].iloc[0]
    ),
    "selected_cv_rmse_se": float(
        pcr_cv_table.loc[pcr_cv_table["n_components"] == selected_n_components,␣
 ↪"cv_rmse_se"].iloc[0]
    )
}])

display(pcr_selection_summary)
display(pcr_cv_table)
```

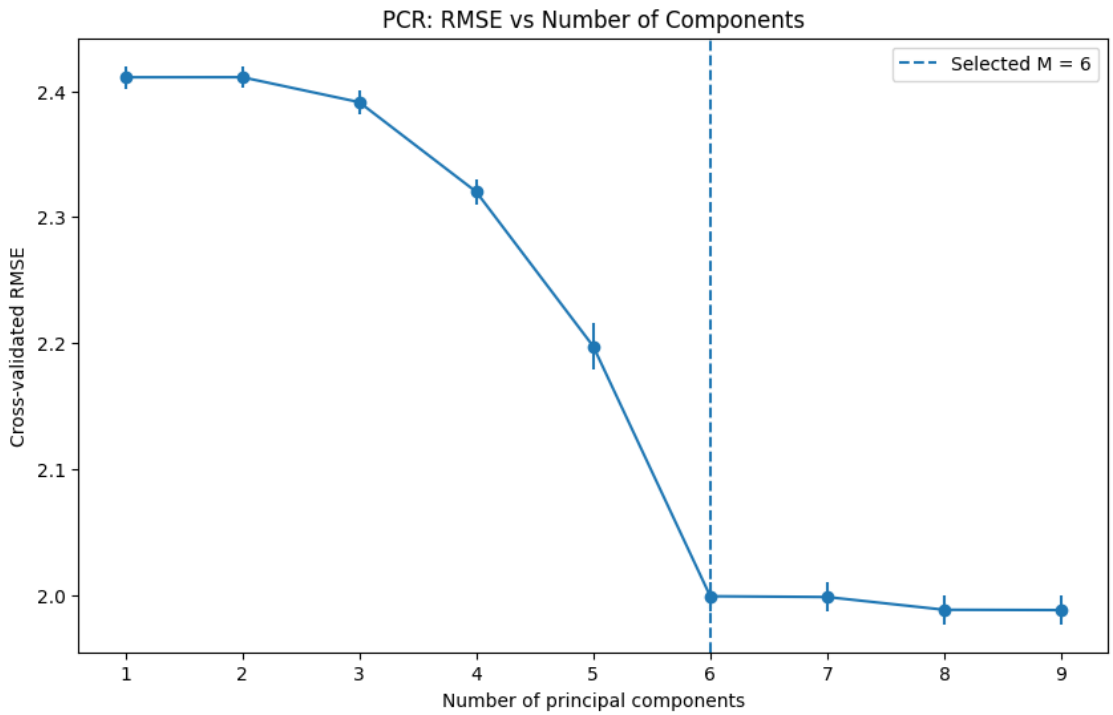
```
                        selection_rule  selected_n_components ␣
↪selected_cv_rmse_mean  selected_cv_rmse_se
```

```
0  10-fold CV + one-standard-error rule                      6                1.
↪998958              0.011579

   n_components  cv_rmse_mean  cv_rmse_se  cv_r2_mean  cv_r2_se
0             1      2.411052    0.008870    0.420994  0.002499
1             2      2.411107    0.008841    0.420968  0.002478
2             3      2.391285    0.009532    0.430449  0.002756
3             4      2.320110    0.009870    0.463834  0.003108
4             5      2.197412    0.018237    0.518883  0.006991
5             6      1.998958    0.011579    0.601993  0.003213
6             7      1.998334    0.011492    0.602242  0.003170
7             8      1.988218    0.011328    0.606262  0.003067
8             9      1.987971    0.011284    0.606359  0.003066
```

12 PCR cross-validation curve

- This plot shows how PCR performance changes as the number of components increases.

[ ]:
```
plt.figure(figsize=(10, 6))
plt.errorbar(
    pcr_cv_table["n_components"],
    pcr_cv_table["cv_rmse_mean"],
    yerr=pcr_cv_table["cv_rmse_se"],
    marker="o"
)
plt.axvline(selected_n_components, linestyle="--", label=f"Selected M =␣
↪{selected_n_components}")
plt.xlabel("Number of principal components")
plt.ylabel("Cross-validated RMSE")
plt.title("PCR: RMSE vs Number of Components")
plt.legend()
plt.show()
```



13 Final PCR model and statistical tables

- This block fits the final PCR model on the training data and generates the full set of infere

[14]:
```
def influence_table_fast(model, top_n=20):
    infl = model.get_influence()

    leverage = np.asarray(infl.hat_matrix_diag)

    # Fast version: internal studentized residuals
    try:
        studentized = np.asarray(infl.resid_studentized_internal)
    except Exception:
        studentized = np.asarray(
            model.resid / np.sqrt(model.mse_resid * (1 - leverage))
        )

    cooks_d = np.asarray(infl.cooks_distance[0])

    n = int(model.nobs)
    p = int(model.df_model) + 1  # include intercept
    leverage_cutoff = 2 * p / n
    cooks_cutoff = 4 / n
```

```
    out = pd.DataFrame({
        "observation": np.arange(n),
        "fitted": np.asarray(model.fittedvalues),
        "residual": np.asarray(model.resid),
        "studentized_residual_internal": studentized,
        "leverage": leverage,
        "cooks_distance": cooks_d,
        "high_leverage_flag": leverage > leverage_cutoff,
        "high_cooks_flag": cooks_d > cooks_cutoff,
        "outlier_flag_abs_studentized_gt_3": np.abs(studentized) > 3
    })

    out = out.sort_values(
        ["cooks_distance", "leverage"],
        ascending=[False, False]
    ).reset_index(drop=True)

    return out.head(top_n), leverage_cutoff, cooks_cutoff


# Final PCR model fit on training data
pcr_pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("pca", PCA(n_components=selected_n_components)),
    ("linreg", LinearRegression())
])

pcr_pipeline.fit(X_train, y_train)

pcr_train_pred = pcr_pipeline.predict(X_train)
pcr_valid_pred = pcr_pipeline.predict(X_valid)

# Extract fitted objects
pcr_scaler = pcr_pipeline.named_steps["scaler"]
pcr_pca = pcr_pipeline.named_steps["pca"]
pcr_linreg = pcr_pipeline.named_steps["linreg"]

# Build component scores on training set and fit statsmodels OLS for␣
 ↪inferential tables
Z_train = pcr_pca.transform(pcr_scaler.transform(X_train))
pc_names = [f"PC{i}" for i in range(1, selected_n_components + 1)]
Z_train_df = pd.DataFrame(Z_train, columns=pc_names)

pcr_sm_model = sm.OLS(
    y_train,
    sm.add_constant(Z_train_df, has_constant="add")
```

```
).fit()

# Tables in PC space
pcr_pc_coef_table = coefficient_table(pcr_sm_model)

pcr_fit_summary = model_fit_summary_table(
    pcr_sm_model,
    f"PCR OLS on selected PCs (M={selected_n_components})"
)

# PCA scores are orthogonal, so VIF should be about 1
if Z_train_df.shape[1] >= 2:
    pcr_pc_vif_table = vif_table(Z_train_df)
else:
    pcr_pc_vif_table = pd.DataFrame({
        "feature": pc_names,
        "vif": [1.0] * len(pc_names)
    })

pcr_assumption_table = assumption_summary_table(
    pcr_sm_model,
    Z_train_df,
    f"PCR OLS on selected PCs (M={selected_n_components})"
)

# Fast influence table
pcr_influence_table, pcr_leverage_cutoff, pcr_cooks_cutoff =␣
 ↪influence_table_fast(
    pcr_sm_model,
    top_n=20
)

# Explained variance table
pcr_explained_variance_table = pd.DataFrame({
    "component": pc_names,
    "explained_variance_ratio": pcr_pca.explained_variance_ratio_,
    "cumulative_explained_variance": np.cumsum(pcr_pca.
 ↪explained_variance_ratio_)
})

# Back-transform PCR coefficients to original predictor scale
beta_pc = pcr_linreg.coef_.reshape(-1, 1)
beta_standardized = (pcr_pca.components_.T @ beta_pc).ravel()
beta_original_scale = beta_standardized / pcr_scaler.scale_

intercept_original_scale = (
    pcr_linreg.intercept_
```

```
    - np.sum((pcr_scaler.mean_ / pcr_scaler.scale_) * beta_standardized)
)

pcr_original_coef_table = pd.DataFrame({
    "term": ["const"] + feature_names,
    "coefficient_original_scale": [intercept_original_scale] +␣
 ↪beta_original_scale.tolist()
})

pcr_metrics_table = pd.concat([
    regression_metrics_table(
        y_train,
        pcr_train_pred,
        f"PCR (M={selected_n_components})",
        "train",
        selected_n_components
    ),
    regression_metrics_table(
        y_valid,
        pcr_valid_pred,
        f"PCR (M={selected_n_components})",
        "validation",
        selected_n_components
    )
], ignore_index=True)

print(f"PCR leverage cutoff = {pcr_leverage_cutoff:.6f}")
print(f"PCR Cook's D cutoff = {pcr_cooks_cutoff:.6f}")

display(pcr_fit_summary)
display(pcr_explained_variance_table)
display(pcr_pc_coef_table)
display(pcr_original_coef_table)
display(pcr_pc_vif_table)
display(pcr_assumption_table)
display(pcr_metrics_table)
display(pcr_influence_table)
```

```
PCR leverage cutoff = 0.000193
PCR Cook's D cutoff = 0.000055

                          model         n_obs  df_model      df_resid          ␣
 ↪ rss  mse_resid  rmse_resid       r2   adj_r2   f_statistic  f_pvalue         ␣
 ↪  aic            bic  durbin_watson  \
0  PCR OLS on selected PCs (M=6) 72,492.000000  6.000000 72,485.000000 289,594.
 ↪997534   3.995240    1.998810 0.602195 0.602162 18,287.905859  0.000000␣
 ↪306,139.324480 306,203.663100       1.994625
```

```
   jarque_bera_stat  jarque_bera_pvalue     skew  kurtosis  condition_number
0    161,959.628500            0.000000 1.450278  9.723609          9.845071

  component  explained_variance_ratio  cumulative_explained_variance
0       PC1                  0.796694                       0.796694
1       PC2                  0.115301                       0.911995
2       PC3                  0.038099                       0.950094
3       PC4                  0.021271                       0.971366
4       PC5                  0.013119                       0.984485
5       PC6                  0.008220                       0.992705

    term  coefficient  std_error      t_value  p_value  ci_lower_95  ci_upper_95
0  const     9.697111   0.007424 1,306.218919 0.000000     9.682561     9.711662
1    PC1     0.767945   0.002772   276.994314 0.000000     0.762511     0.773379
2    PC2     0.007708   0.007288     1.057650 0.290219    -0.006576     0.021992
3    PC3    -0.526829   0.012678   -41.554875 0.000000    -0.551677    -0.501980
4    PC4     1.321651   0.016967    77.894952 0.000000     1.288395     1.354906
5    PC5     2.180109   0.021605   100.908949 0.000000     2.137764     2.222454
6    PC6     3.345512   0.027295   122.570060 0.000000     3.292014     3.399009

             term  coefficient_original_scale
0           const                    3.775713
1          length                    1.117587
2        diameter                    4.308959
3          height                   20.525998
4    whole_weight                    0.147606
5  whole_weight_1                  -11.949376
6  whole_weight_2                   -4.719046
7    shell_weight                   25.984724
8           sex_I                   -0.745887
9           sex_M                   -0.032191

  feature      vif
0     PC5 1.000000
1     PC2 1.000000
2     PC6 1.000000
3     PC3 1.000000
4     PC1 1.000000
5     PC4 1.000000

                           model                   assumption                   ␣
↪check        value              guideline  result
0  PCR OLS on selected PCs (M=6)                   Linearity       RESET test␣
↪p-value     0.000000  p > 0.05 is desirable  Review
1  PCR OLS on selected PCs (M=6)                   Linearity     Rainbow test␣
↪p-value     0.974665  p > 0.05 is desirable    Pass
2  PCR OLS on selected PCs (M=6)      Independence of errors  Durbin-Watson␣
↪statistic     1.994625  Around 2 is desirable    Pass
3  PCR OLS on selected PCs (M=6)            Homoscedasticity    Breusch-Pagan␣
↪p-value     0.000000  p > 0.05 is desirable  Review
```

```
4  PCR OLS on selected PCs (M=6)      Normality of residuals      Jarque-Bera␣
↪p-value     0.000000  p > 0.05 is desirable  Review
5  PCR OLS on selected PCs (M=6)      Normality of residuals     Shapiro-Wilk␣
↪p-value     0.000000  p > 0.05 is desirable  Review
6  PCR OLS on selected PCs (M=6)                Collinearity              ␣
↪Maximum VIF     1.000000   Below 5 is desirable    Pass
7  PCR OLS on selected PCs (M=6)    Influential observations     Count Cook's D␣
↪> 4/n 4,517.000000   Smaller is desirable  Review
8  PCR OLS on selected PCs (M=6)  High leverage observations    Count leverage >␣
↪2p/n 4,367.000000   Smaller is desirable  Review

      model       split  n_obs  n_predictors            rss      mse     rmse  ␣
↪   mae       r2   adj_r2
0  PCR (M=6)       train  72492             6 289,594.997534 3.994855 1.998713 1.
↪373681 0.602195 0.602162
1  PCR (M=6)  validation  18123             6  75,242.643885 4.151776 2.037591 1.
↪399946 0.595830 0.595696

   observation    fitted   residual  studentized_residual_internal  leverage ␣
↪cooks_distance  high_leverage_flag  high_cooks_flag ␣
↪outlier_flag_abs_studentized_gt_3
0         10362 29.625024 -21.625024                     -11.230943  0.072022  ␣
↪    1.398492                True             True                              ␣
↪ True
1         60373 29.748183 -19.748183                     -10.234743  0.068125  ␣
↪    1.093974                True             True                              ␣
↪ True
2         53381 26.979296 -18.979296                      -9.748845  0.051339  ␣
↪    0.734763                True             True                              ␣
↪ True
3         60496 28.035514 -19.035514                      -9.535147  0.002457  ␣
↪    0.031995                True             True                              ␣
↪ True
4         36375 20.453405 -10.453405                      -5.248353  0.007052  ␣
↪    0.027947                True             True                              ␣
↪ True
5         53748 20.626655 -10.626655                      -5.333192  0.006253  ␣
↪    0.025567                True             True                              ␣
↪ True
6         66670 21.842940  -9.842940                      -4.942208  0.007193  ␣
↪    0.025282                True             True                              ␣
↪ True
7         27584 18.937743  -7.937743                      -3.986526  0.007657  ␣
↪    0.017517                True             True                              ␣
↪ True
```

```
8         18468 24.097160 -15.097160                      -7.558220  0.001361  ␣
↪    0.011122                True             True                              ␣
↪ True
9         25431 23.482911 -12.482911                      -6.249910  0.001516  ␣
↪    0.008470                True             True                              ␣
↪ True
10        46525  9.195451  19.804549                       9.910642  0.000499  ␣
↪    0.006999                True             True                              ␣
↪ True
11         2538 22.956989 -11.956989                      -5.985861  0.001271  ␣
↪    0.006516                True             True                              ␣
↪ True
12        53920 24.405050 -11.405050                      -5.709730  0.001334  ␣
↪    0.006220                True             True                              ␣
↪ True
13        56911 22.162603 -10.162603                      -5.087473  0.001236  ␣
↪    0.004577                True             True                              ␣
↪ True
14        67534 21.529866 -10.529866                      -5.270966  0.001099  ␣
↪    0.004368                True             True                              ␣
↪ True
15         2778 21.131844 -10.131844                      -5.071946  0.001186  ␣
↪    0.004362                True             True                              ␣
↪ True
16         1404 22.177305  -9.177305                      -4.594547  0.001376  ␣
↪    0.004156                True             True                              ␣
↪ True
17        35575 21.970607  -8.970607                      -4.491201  0.001436  ␣
↪    0.004144                True             True                              ␣
↪ True
18        41219 25.567963  -8.567963                      -4.289785  0.001516  ␣
↪    0.003990                True             True                              ␣
↪ True
19        50019 21.030013 -11.030013                      -5.520509  0.000804  ␣
↪    0.003501                True             True                              ␣
↪ True
```

14 Faster diagnostic plots for PCR model

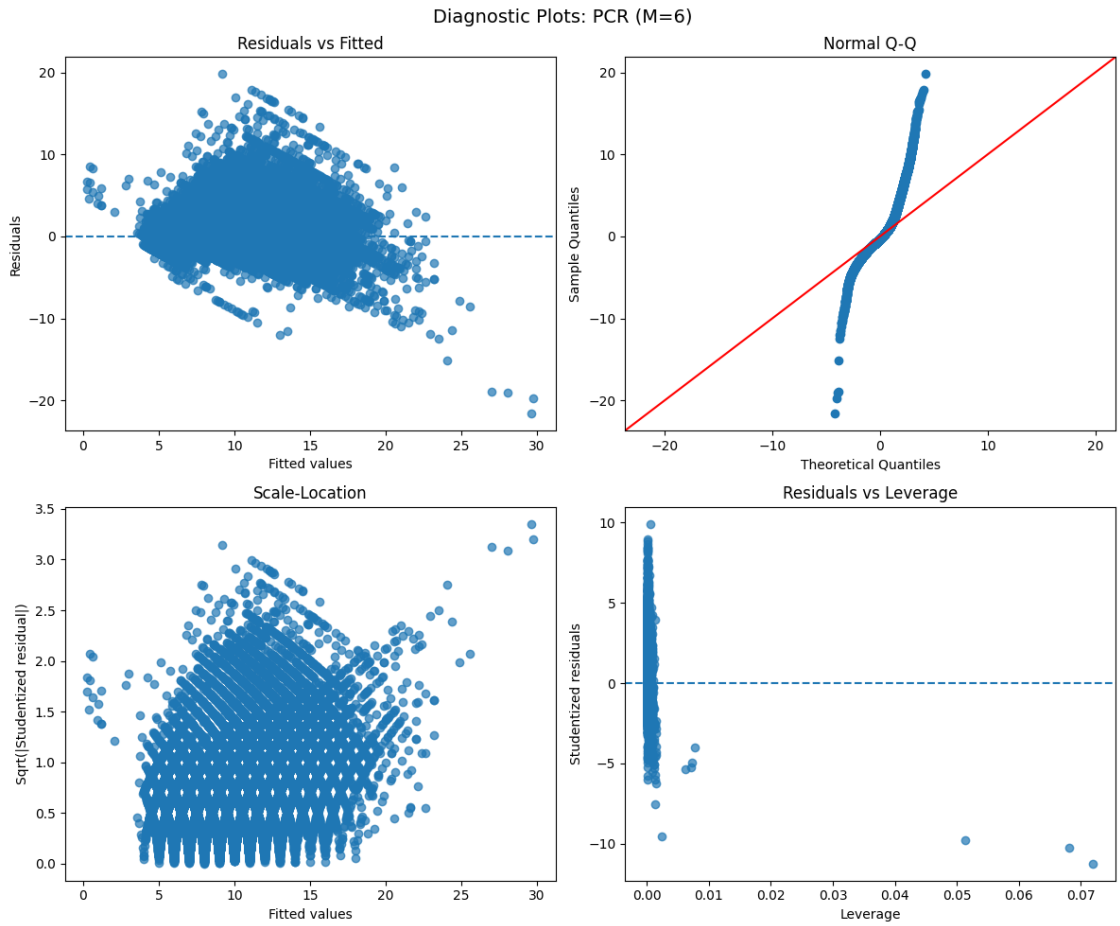
- This block plays the same role as the subset-model diagnostic plot block, but for PCR.

[15]:
```
def plot_ols_diagnostics_fast(model, title):
    fitted = np.asarray(model.fittedvalues)
    resid = np.asarray(model.resid)

    infl = model.get_influence()
```

```
    leverage = np.asarray(infl.hat_matrix_diag)

    # Fast version: internal studentized residuals
    try:
        studentized = np.asarray(infl.resid_studentized_internal)
    except Exception:
        studentized = resid / np.sqrt(model.mse_resid * (1 - leverage))

    fig, axes = plt.subplots(2, 2, figsize=(12, 10))

    # 1. Residuals vs Fitted
    axes[0, 0].scatter(fitted, resid, alpha=0.7)
    axes[0, 0].axhline(0, linestyle="--")
    axes[0, 0].set_title("Residuals vs Fitted")
    axes[0, 0].set_xlabel("Fitted values")
    axes[0, 0].set_ylabel("Residuals")

    # 2. Normal Q-Q
    sm.qqplot(resid, line="45", ax=axes[0, 1])
    axes[0, 1].set_title("Normal Q-Q")

    # 3. Scale-Location
    axes[1, 0].scatter(fitted, np.sqrt(np.abs(studentized)), alpha=0.7)
    axes[1, 0].set_title("Scale-Location")
    axes[1, 0].set_xlabel("Fitted values")
    axes[1, 0].set_ylabel("Sqrt(|Studentized residual|)")

    # 4. Residuals vs Leverage
    axes[1, 1].scatter(leverage, studentized, alpha=0.7)
    axes[1, 1].axhline(0, linestyle="--")
    axes[1, 1].set_title("Residuals vs Leverage")
    axes[1, 1].set_xlabel("Leverage")
    axes[1, 1].set_ylabel("Studentized residuals")

    fig.suptitle(title, fontsize=14)
    plt.tight_layout()
    plt.show()


print(f"PCR leverage cutoff = {pcr_leverage_cutoff:.6f}")
print(f"PCR Cook's D cutoff = {pcr_cooks_cutoff:.6f}")

plot_ols_diagnostics_fast(
    pcr_sm_model,
    f"Diagnostic Plots: PCR (M={selected_n_components})"
)
```

```
PCR leverage cutoff = 0.000193
```

```
PCR Cook's D cutoff = 0.000055
```



15 Complete comparison table

- This block brings all the important models together for side-by-side comparison.

[16]:
```
subset_selected_cv_rmse = float(selected_subset_row["cv_rmse_mean"])
pcr_selected_cv_rmse = float(
    pcr_cv_table.loc[pcr_cv_table["n_components"] == selected_n_components,␣
 ↪"cv_rmse_mean"].iloc[0]
)

model_comparison_table = pd.concat([
    baseline_comparison,
    regression_metrics_table(
        y_valid, subset_valid_pred,
        "Regularization-informed best subset OLS", "validation",
        len(selected_subset_vars)
```

```
    ),
    regression_metrics_table(
        y_valid, pcr_valid_pred,
        f"PCR (M={selected_n_components})", "validation",
        selected_n_components
    )
], ignore_index=True)

extra_summary_rows = pd.DataFrame([
    {
        "model": "Regularization-informed best subset OLS",
        "selected_structure": " + ".join(selected_subset_vars),
        "train_r2": float(subset_model.rsquared),
        "train_adj_r2": float(subset_model.rsquared_adj),
        "train_aic": float(subset_model.aic),
        "train_bic": float(subset_model.bic),
        "selected_cv_rmse": subset_selected_cv_rmse
    },
    {
        "model": f"PCR (M={selected_n_components})",
        "selected_structure": f"{selected_n_components} principal components",
        "train_r2": float(pcr_sm_model.rsquared),
        "train_adj_r2": float(pcr_sm_model.rsquared_adj),
        "train_aic": float(pcr_sm_model.aic),
        "train_bic": float(pcr_sm_model.bic),
        "selected_cv_rmse": pcr_selected_cv_rmse
    }
])

display(model_comparison_table)
display(extra_summary_rows)
```

```
                                    model       split  n_obs  n_predictors      ␣
↪      rss       mse     rmse      mae        r2    adj_r2
0                           Null mean model  validation  18123             0␣
↪186,165.952688 10.272358 3.205052 2.349198 -0.000000 -0.000000
1              Full OLS main-effects model  validation  18123             9 ␣
↪74,219.532043  4.095323 2.023690 1.393015  0.601326  0.601128
2  Regularization-informed best subset OLS  validation  18123             5 ␣
↪75,281.179636  4.153903 2.038113 1.400566  0.595623  0.595512
3                                PCR (M=6)  validation  18123             6 ␣
↪75,242.643885  4.151776 2.037591 1.399946  0.595830  0.595696

                                    model                                 ␣
↪selected_structure  train_r2  train_adj_r2      train_aic      train_bic ␣
↪selected_cv_rmse
```

```
0  Regularization-informed best subset OLS  shell_weight + whole_weight_1 +␣
 ↪height + diame…  0.602080      0.602052 306,158.333093 306,213.480482          ␣
 ↪1.999227
1                                PCR (M=6)                                  6␣
 ↪principal components  0.602195      0.602162 306,139.324480 306,203.663100     ␣
 ↪      1.998958
```

16 Recommended model based on validation RMSE

- This block takes the assignment models only, the regularization-informed subset model and the

[18]:
```
comparison_for_choice = model_comparison_table.loc[
    model_comparison_table["model"].isin([
        "Regularization-informed best subset OLS",
        f"PCR (M={selected_n_components})"
    ])
].copy()

recommended_model_name = comparison_for_choice.sort_values("rmse").
 ↪iloc[0]["model"]

print("Recommended submission model based on validation RMSE:")
print(recommended_model_name)
```

```
Recommended submission model based on validation RMSE:
PCR (M=6)
```

17 Refit final chosen structures on the full training data

- Once model selection is complete, this block refits the final model structures using all avai

[19]:
```
# Final subset model on full data using selected subset from training-only␣
 ↪selection
subset_model_full = sm.OLS(
    y_full,
    sm.add_constant(X_full[selected_subset_vars], has_constant="add")
).fit()

subset_test_pred = subset_model_full.predict(
    sm.add_constant(X_test_full[selected_subset_vars], has_constant="add")
)
subset_test_pred = np.clip(subset_test_pred, 0, None)

# Final PCR model on full data using selected number of components from␣
 ↪training-only selection
pcr_pipeline_full = Pipeline([
    ("scaler", StandardScaler()),
```

```
    ("pca", PCA(n_components=selected_n_components)),
    ("linreg", LinearRegression())
])

pcr_pipeline_full.fit(X_full, y_full)
pcr_test_pred = pcr_pipeline_full.predict(X_test_full)
pcr_test_pred = np.clip(pcr_test_pred, 0, None)

print("Refit complete.")
```

```
Refit complete.
```

18 Export statistical tables workbook

This final block saves the analysis results into one Excel workbook.

[ ]:
```
from pathlib import Path

output_dir = Path(r"F:
 ↪\NU\predictive_analysis\week2\assignment2\playground-series-s4e4\week3")
output_dir.mkdir(parents=True, exist_ok=True)

tables_to_export = {
    "lasso_cv_table": lasso_cv_table,
    "lasso_screening": lasso_screening_table,
    "lasso_screening_summary": lasso_screening_summary,
    "subset_search_all": subset_search_table,
    "subset_frontier": subset_frontier_table,
    "subset_selection_summary": subset_selection_summary,
    "subset_coefficients": subset_coef_table,
    "subset_fit_summary": subset_fit_summary,
    "subset_vif": subset_vif_table,
    "subset_assumptions": subset_assumption_table,
    "subset_metrics": subset_metrics_table,
    "subset_influence_top20": subset_influence_table,
    "pcr_cv_table": pcr_cv_table,
    "pcr_selection_summary": pcr_selection_summary,
    "pcr_explained_variance": pcr_explained_variance_table,
    "pcr_pc_coefficients": pcr_pc_coef_table,
    "pcr_original_coeffs": pcr_original_coef_table,
    "pcr_fit_summary": pcr_fit_summary,
    "pcr_vif": pcr_pc_vif_table,
    "pcr_assumptions": pcr_assumption_table,
    "pcr_metrics": pcr_metrics_table,
    "pcr_influence_top20": pcr_influence_table,
    "baseline_comparison": baseline_comparison,
    "model_comparison": model_comparison_table,
```

```
    "extra_model_summary": extra_summary_rows
}

for table_name, df in tables_to_export.items():
    csv_path = output_dir / f"{table_name}.csv"
    df.to_csv(csv_path, index=False)

print(f"All CSV files were saved to: {output_dir}")
```

```
All CSV files were saved to:
F:\NU\predictive_analysis\week2\assignment2\playground-series-s4e4\week3
```